

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Сбалансированные двоичные деревья поиска

Практикум

по выполнению лабораторной работы № 30

по дисциплине "Структуры и алгоритмы обработки данных"

Ростов-на-Дону

20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.

Практикум по выполнению лабораторной работы по дисциплине "Структуры и алгоритмы обработки данных". ДГТУ, г. Ростов на-Дону, 20 г.

В практикуме кратко изложены теоретические вопросы, необходимые для успешного выполнения лабораторной работы, рабочее задание и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлениям 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный за выпуск:

зав. кафедрой Долгов В.В.

Издательский центр ДГТУ, 20

Оглавление

Введение 4

Теоретическая часть 4

1. Базовые концепции 4

2. Рандомизированные деревья поиска 5

3. AVL-деревья 9

4. Красно-черные деревья 11

Инструкция по технике безопасности при проведении работы 15

Цель работы 15

Рабочее задание 15

Общие положения 17

Материально-техническое обеспечение работы 17

Порядок выполнения работы 18

Контрольные вопросы 18

Перечень использованных информационных ресурсов 20

Введение

Двоичные деревья поиска, реализованные так, что новые узлы всегда добавляются как листья не всегда организованы оптимальным образом, то есть не всегда обеспечивают наилучшую – логарифмическую эффективность операции поиска элемента. Наибольшее количество операций сравнения ключей, которые необходимо выполнить в процессе поиска, определяет его эффективность и равняется высоте дерева. Если добавляются уже упорядоченные данные, высота дерева становится равной количеству элементов в нем, что приводит к линейной зависимости количества необходимых в среднем и худшем случаях операций сравнения.

Деревья поиска с наименьшей возможной для заданного количества узлов высотой называют сбалансированными.

Возникает задача создания сбалансированных деревьев поиска, в которых операции добавления нового узла и удаления заданного узла не приводили бы к получению несбалансированного дерева.

Теоретическая часть

1. Базовые концепции

В том случае, если для каждого узла двоичного дерева поиска выполняется условие, что количество узлов в его левом поддереве может отличаться от количества узлов в его правом поддереве не более чем на единицу, дерево называют идеально сбалансированным. Условие идеальной сбалансированности гарантирует возможность создания алгоритма поиска с логарифмической сложностью. Всякое двоичное дерево поиска можно преобразовать в идеально сбалансированное. Например, можно выполнить симметричный обход, получить значения узлов дерева в виде упорядоченной линейной последовательности и, затем, построить идеально сбалансированное

дерево добавляя в него значения из упорядоченной линейной последовательности в качестве листов в нужном порядке, то есть вначале добавляем средний элемент, затем рекурсивно повторяем добавление для левой и правой половин до тех пор, пока не добавим все элементы. Эта схема плоха тем, что требует полной перестройки дерева. Такое преобразование в общем случае нужно выполнять после каждого добавления или удаления элемента. Очевидно, что это слишком затратно, то есть неэффективно. Необходимо найти решение, которое обеспечивало бы не только эффективный поиск, но также и эффективные реализации операций добавления и удаления элементов. Для этого можно использовать разные приемы. Один из них заключается в том, чтобы ослабить требование по сбалансированности дерева. Пусть дерево будет не идеально, но все же сбалансировано так, чтобы эффективность поиска оставалась логарифмической, но худшей чем в идеальном случае (например, вдвое), если при этом операции добавления и удаления потребуют дополнительных действий по балансировке дерева, требующих лишь локальной его перестройки (изменения затрагивают только некоторые узлы, но не требуют перестройки всего дерева) то все операции – поиска, добавления и удаления окажутся достаточно эффективными.

2. Рандомизированные деревья поиска

Если новые узлы добавляются в двоичное дерево поиска всегда в качестве листов, но при этом значения их ключей задаются в случайном порядке (и распределены равномерно), то дерево оказывается в среднем достаточно хорошо сбалансированным. Этот факт используется в рандомизированных двоичных деревьях поиска. В каком бы порядке на самом деле не добавлялись элементы, операция добавления узла в таких деревьях

использует генератор случайных чисел для определения позиции в дереве, которую должен занять новый узел. То есть новый узел может стать как листом, так и корнем, или внутренним узлом дерева (корнем какого-то поддерева). Для этого нужно реализовать операцию добавления нового узла в качестве корня дерева.

Чтобы смоделировать случайное добавление нового узла мы должны предположить, что он с равной вероятностью может оказаться в любом узле дерева. Если в дереве n узлов, то вероятность размещения узла в корне, как и в любой другой позиции, равна 1/n. Так как добавляется новый узел, в дереве теперь будет n+1 узел и тогда для нового узла вероятность стать корнем равна 1/(n+1). Если добавление производится не в корень всего дерева, но в корень какого-то его поддерева, очевидно, что n будет задавать число узлов в этом поддереве (включая его корень). Таким образом, в каждом узле рандомизированного двоичного дерева поиска может храниться, помимо ключа key, еще и количество узлов n в том поддереве, для которого данный узел является корнем.

```
record RBSTNode
    key : T
    n : integer
    leftChild, rightChild, parent : RBSTNode^
```

здесь RBSTNode^ обозначает ссылку на узел рандомизированного двоичного дерева поиска. А само рандомизированное двоичное дерево будет представлено типом

```
record RBST
    root : RBSTNode^
```

Собственно добавление нового ключа (value) в рандомизированное двоичное дерево поиска можно описать следующей схемой:

1. Создать новый узел, задать для него key = value, задать n равным единице.

2. Если дерево пустое, сделать новый узел корнем дерева. Выход.

3. Иначе. Сгенерировать случайное число x в диапазоне [0..1). Получить n узла, указанного как корень – root.n.

4. Если x < 1/(root.n + 1) добавить новый узел в качестве корня (вспомогательный алгоритм AddRoot).

5. Иначе. Если value < root.key вызвать этот же алгоритм добавления узла рекурсивно, передав root.left в качестве root.

6. Иначе, в рекурсивном вызове в качестве root передать root.right.

7. В любом случае увеличить на 1 root.n.

Для вспомогательного алгоритма AddRoot – добавление узла в качестве корня дерева удобно использовать операции вращения (поворота), позволяющие поднять какие-то узлы в дереве на один уровень выше (опустив при этом некоторые другие узлы на уровень ниже).

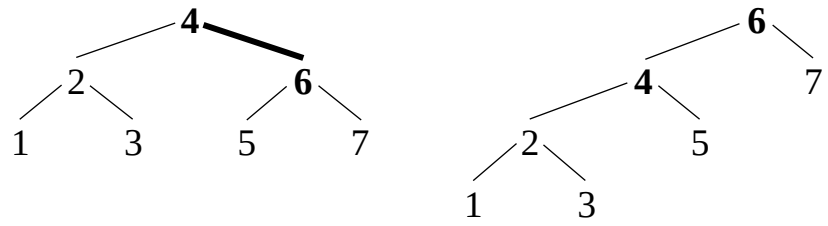


Рис.1. Правый поворот в RBST.

Например, поворот вправо для узла 4 на Рис.1 можно представить себе как поворот связи этого узла с его правым потомком (6). При этом 4 и его левое поддерево опускаются на один уровень вниз, 6 и его правое поддерево

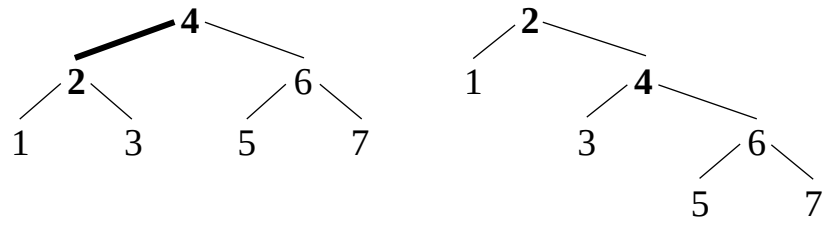


Рис.2. Левый поворот в RBST.

поднимаются на один уровень вверх, а левое поддерево 6 (5) становится правым поддеревом 4. При этом узел 6 становится корнем всего дерева, значит n для него должно теперь содержать значение n узла 4 (бывшего корня), а новое значение n для 4 должно стать равным сумме количества узлов в его левом поддереве, количества узлов в бывшем левом поддереве 6 (а теперь правом поддереве 4) и единицы (сам узел 4).

При левом повороте (Рис.2) мы начинаем с рассмотрения узла, для которого выполняется поворот (4) и его левого потомка (2). При этом 4 и его правое поддерево опускаются на один уровень вниз, 2 и его левое поддерево поднимаются на один уровень вверх, а правое поддерево 2 (3) становится левым поддеревом 4. n узла 2 должен стать равным n узла 4, а n узла 4 – сумме значений n его правого поддерева, n его нового левого поддерева и единицы.

Таким образом, правый поворот для корня дерева, делает новым корнем дерева корень правого поддерева, а левый поворот – корень левого поддерева, причем после поворотов дерево остается двоичным деревом поиска.

Теперь можем добавление корня (AddRoot) выразить через повороты:

1. Создать новый узел, задать для него key = value, задать n равным единице.
2. Если дерево пустое, сделать новый узел корнем дерева. Выход.
3. Иначе. Если value < root.key вызвать этот же алгоритм AddRoot рекурсивно, передав root.left в качестве root. Выполнить правый поворот для root.
4. Иначе, в рекурсивном вызове в качестве root передать root.right и выполнить правый поворот для root.

Очевидно, что алгоритм операции поиска элемента остается таким же, как в не рандомизированном двоичном дереве поиска.

3. AVL-деревья

AVL-дерево – это двоичное дерево поиска сбалансированное по высоте. У каждого узла высота левого поддерева должна отличаться от высоты правого поддерева не более чем на единицу. С каждым узлом дерева связан дополнительный параметр, баланс – разность высот его поддеревьев. Из определения AVL-дерева следует, что параметр баланса может иметь значение -1, 0 или 1. Далее рассматривается вариант, в котором баланс – это разность высот левого и правого поддеревьев. Пример AVL-дерева приведен на Рис.3.

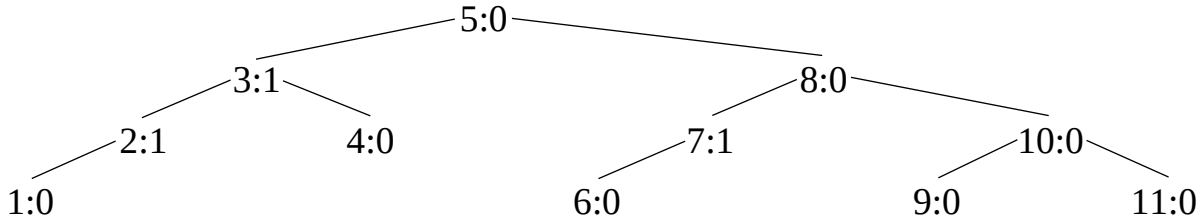


Рис.3. AVL-дерево. После значка : указано значение баланса.

AVL-дерево является самобалансирующимся, при выполнении любой операции, изменяющей дерево (добавление/удаление узла) дерево перестраивается (балансируется) так, чтобы оно осталось AVL-деревом.

Балансировку дерева можно выполнять, применяя локальные операции – повороты. Если после добавления нового элемента в некотором узле баланс нарушается, то есть становится равным 2 или -2, нужно выполнить поворот для

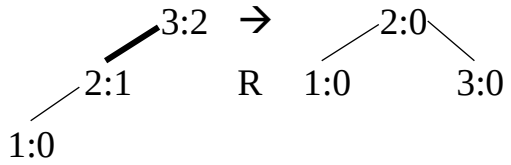


Рис.4. Правый поворот (R)

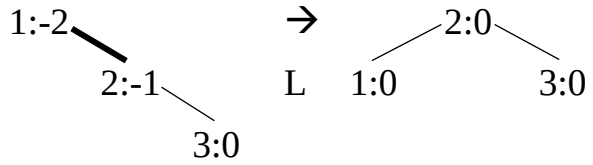


Рис.5. Левый поворот (L)

этого узла и, быть может, его потомка. Всего возможны четыре вида поворотов - правый (Рис.4), левый (Рис.5), левый-правый (Рис.6) и правый-левый (Рис.7).

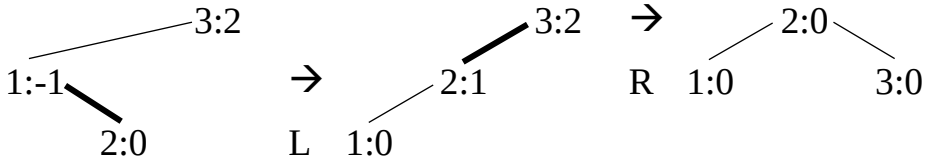


Рис.6. Левый-правый поворот (LR). Левый для 1, затем правый для 3

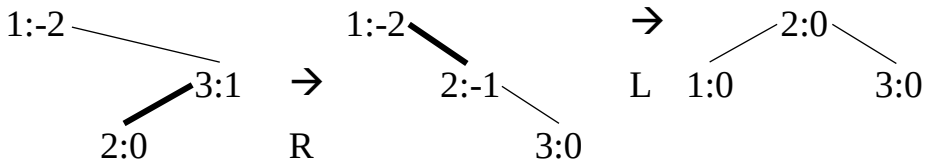


Рис.7. Правый-левый поворот (RL). Правый для 3, затем левый для 1

Из рисунков 4-7 можно увидеть, что правый поворот выполняется для узла, в котором баланс стал равен 2, если при этом баланс его левого потомка равен 1, левый поворот – при нарушенном балансе -2, если у правого потомка баланс равен -1, левый-правый – при нарушенном балансе 2, если у левого потомка баланс -1 и правый-левый – при нарушенном балансе -2, если у правого потомка баланс 1.

Схема добавления нового ключа со значением value в AVL-дерево:

1. Создать новый узел. Задать для его ключа значение value. Баланс положить равным нулю.

2. Добавить новый узел как лист, обновляя на обратном ходе рекурсии значения параметра баланса для всех пройденных узлов.

3. Проверить баланс у родителя добавленного узла. Если он правильный продолжать проверять для "родителя родителя" и так вплоть до корня дерева.

4. Если баланс нарушен выполнить один из поворотов как описано выше, обновляя баланс узлов.

Пример, добавление узла 6 в AVL-дерево на Рис.8.

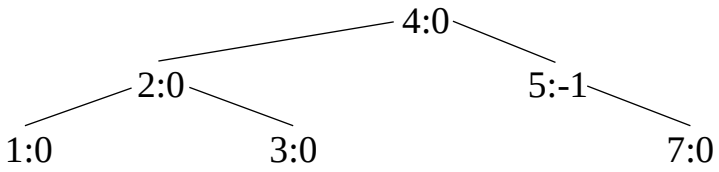


Рис.8. Исходное AVL-дерево.

После добавления 6 в качестве листа (Рис.9) нарушается баланс 5.

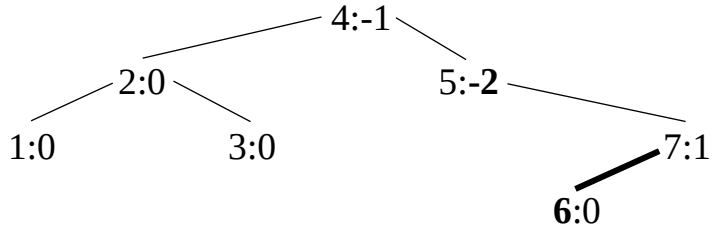


Рис.9. Добавление листа.

Получился вариант балансов в узле 5 и его правом потомке: -2 и 1. Нужен правый-левый поворот для 5. Вначале правый поворот для правого потомка 5 (7) - Рис.10.

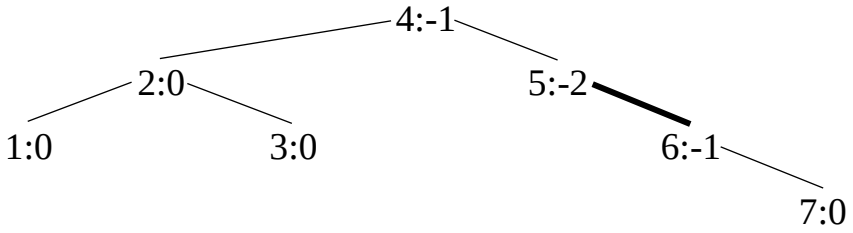


Рис.10. Результат правого поворота для 7.

Затем левый поворот для 5 – Рис.11.

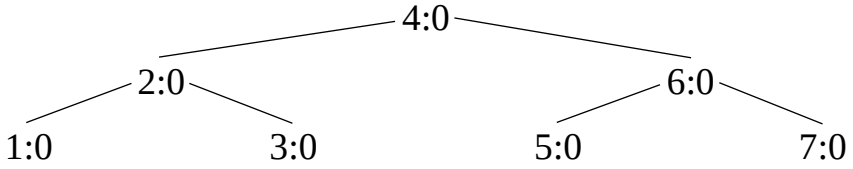


Рис.11. Результат левого поворота для 5.

4. Красно-черные деревья

Красно-черное дерево – это самобалансирующееся двоичное дерево поиска, для узлов которого должны выполняться следующие требования: с каждым узлом связан дополнительный атрибут – цвет, узел может быть либо красным либо черным, корень дерева черный, у каждого листа пустые ссылки на потомков рассматриваются как "пустые" (null) узлы и они черные, у красных

узлов потомки черные (как следствие и предок красного узла черный), у каждого пути от узла до его (пустого) листа одинаковое количество черных узлов (длины черных путей одинаковые или, другими словами, одинакова черная высота).

Пример красно-черного дерева приведен на рис.12. Черными кружками

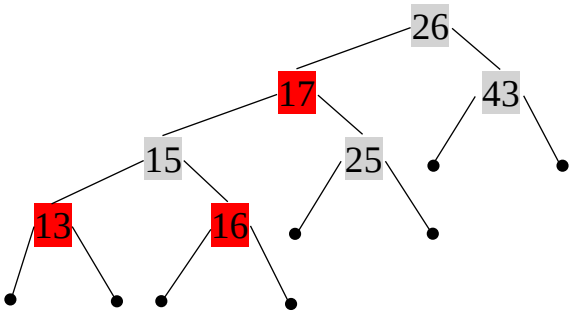
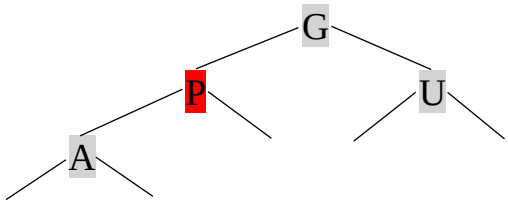


Рис.12. Красно-черное дерево



A – текущий узел, P – родитель, U – дядя, G – дедушка

обозначены пустые (null) узлы-листья. По аналогии с родственными отношениями между людьми, мы можем для некоторого узла A (см. рисунок выше) назвать его узел-предок P родительским, предка родителя G "дедушкой", а другого потомка "дедушки", расположенного на том же уровне дерева что и родительский - U "дядей".

Тогда процесс добавления нового узла в красно-черное дерево формально можно описать следующим образом:

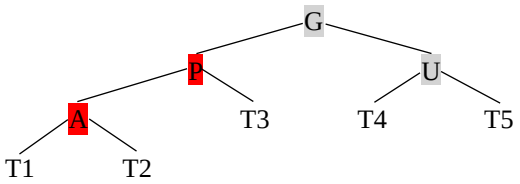
1. Добавить новый узел как лист считая его красным.

2. Если это корень, поменять его цвет на черный и завершить процедуру добавления.

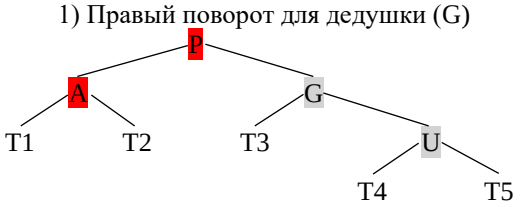
3. Если не корень, проверить цвет родительского узла, если он черный – не изменять его цвет, если же красный – проверить цвет "дяди".

4. Если "дядя" красный – меняем цвета и родителя, и "дяди" на черный, а "дедушки" на красный. Повторяем процедуру для "дедушки". Если "дедушка" корень, не меняем его цвет на красный.

5. Если "дядя" черный нужно рассмотреть четыре случая и, в зависимости от случая: правый-правый (RR), левый-правый (LR), левый-левый (LL) или правый-левый (RL), выполнить поворот и, быть может, изменить цвета "дедушки" и "родителя".



1) Правый поворот для дедушки (G)



2) Обмен цветов дедушки (G) и родителя (P)

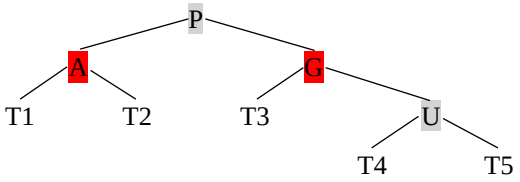
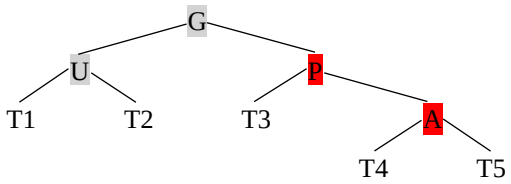
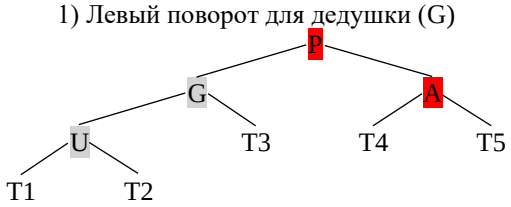


Рис.13. После добавления узла A. LL-поворот.



1) Левый поворот для дедушки (G)



2) Обмен цветов дедушки (G) и родителя (P)

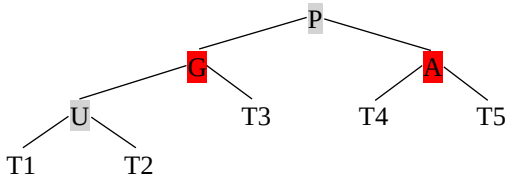
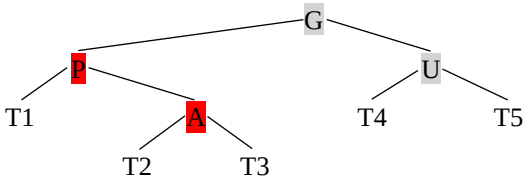
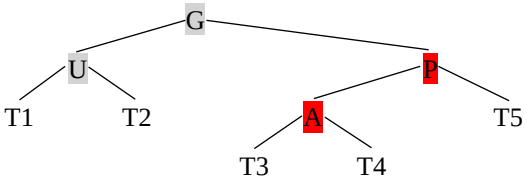


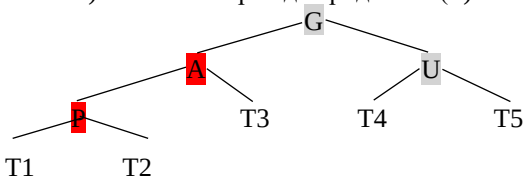
Рис.14. После добавления узла A. RR-поворот.



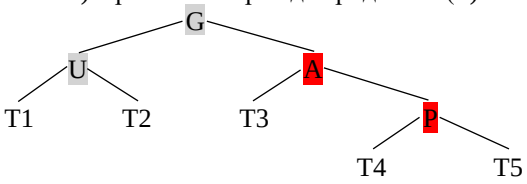
1) Левый поворот для родителя (P)



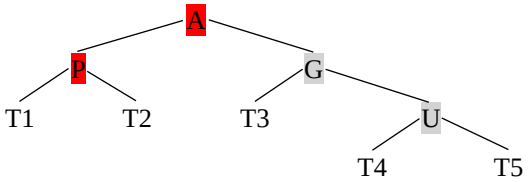
1) Правый поворот для родителя (P)



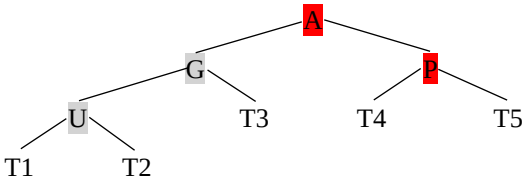
2) LL-поворот: Правый поворот для дедушки (G)



2) RR-поворот: Левый поворот для дедушки (G



3) Обмен цветов дедушки (G) и родителя (A)



3) Обмен цветов дедушки (G) и родителя (A)

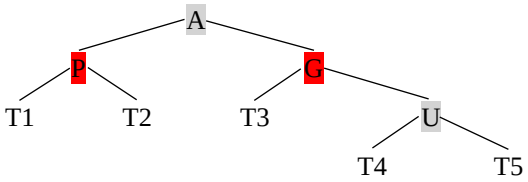


Рис.15. После добавления узла A. LR-поворот.

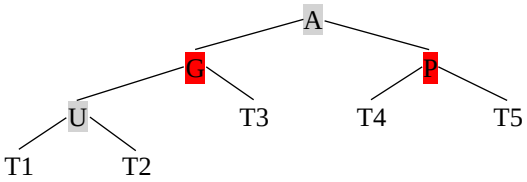


Рис.16. После добавления узла A. RL-поворот.

Например, добавим 14 в красно-черное дерево. Шаг 1. Добавляем как лист (Рис.17). Следуем описанной выше схеме. Это не корень. Родитель (13) красный, дядя (16) красный – шаг 2, меняем цвета родителя и дяди на черный, дедушки на красный (Рис.18). Повторяем для дедушки (15).

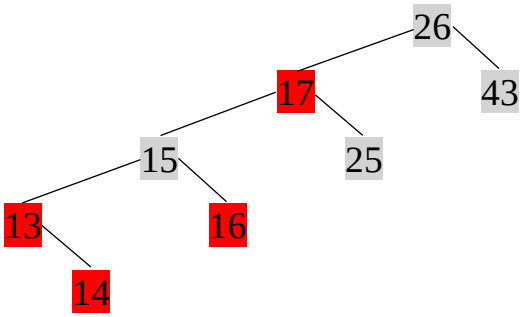


Рис.17. Добавление 14. Шаг 1.

Его родитель (17) красный, дядя (43) черный. Получили вариант "левый-левый" – LL (родитель 17 левый для своего родителя, и сам 15 левый для родителя 17).

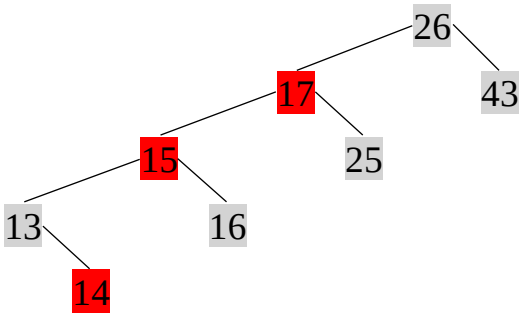


Рис.18. Добавление 14. Шаг 2.

Нужно выполнить шаг 3 - правый поворот для дедушки (26), затем шаг 4 - обменять цвета дедушки (26) и родителя (17) – Рис.19.

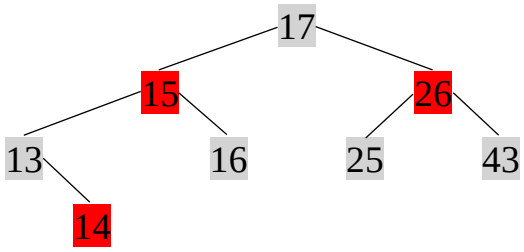


Рис.19. Добавление 14. Шаги 3,4.

Инструкция по технике безопасности при проведении работы

При выполнении работы нужно выполнять требования по технике безопасности при работе с электронными приборами.

Цель работы

Изучение сбалансированных двоичных деревьев поиска. Ознакомление с подходами, реализованными в рандомизированных, AVL- и красно-черных деревьях для выполнения самобалансировки деревьев при вставке узлов.

Рабочее задание

ЗАДАНИЕ 1. Реализуйте структуру данных "Рандомизированное двоичное дерево поиска". Операции: проверки на пустоту, добавления нового узла,

добавления узла в качестве корня дерева, поиска узла с заданным ключом (возвращает ссылку на узел), поворотов. Можно ограничиться целыми значениями ключей.

ЗАДАНИЕ 2. Создайте реализацию структуры данных "AVL-дерево. Операции: проверки на пустоту, добавления нового узла, поиска узла с заданным ключом (возвращает ссылку на узел), поворотов.

ЗАДАНИЕ 3. Создайте реализацию структуры данных "Красно-черное дерево". Операции: проверки на пустоту, добавления нового узла, поиска узла с заданным ключом (возвращает ссылку на узел), поворотов, перекраски узлов (опционально).

ЗАДАНИЕ 4. Создайте реализацию структуры данных "Множество" (SetR) для работы с целыми числами на основе структуры данных "Рандомизированное дерево".

ЗАДАНИЕ 5. Создайте реализацию структуры данных "Множество" (SetAVL) для работы с целыми числами на основе структуры данных "AVL-дерево".

ЗАДАНИЕ 6. Создайте реализацию структуры данных "Множество" (SetRB) для работы с целыми числами на основе структуры данных "Красно-черное дерево".

ЗАДАНИЕ 7. Реализуйте алгоритм для тестирования реализаций структуры данных "Множество" на основе структур данных "Двоичное дерево поиска " из предыдущей лабораторной работы, "Рандомизированное дерево", "AVL-дерево", "Красно-черное дерево". Нужно сравнить время выполнения операции поиска элемента для деревьев разного размера, используя при добавлении элементов сгенерированные случайным образом значения ключей. Чтобы получить надежное значение, необходимо каждую операцию поиска повторить

многократно (в цикле), определить время выполнения всех повторов и, затем, найти среднее.

ЗАДАНИЕ 8. Реализуйте операции проверки заданного двоичного дерева поиска на то, является ли оно идеально сбалансированным, AVL, красно-черным деревом.

Общие положения

Сбалансированные деревья поиска используются во многих задачах, в которых критически важным является поиск нужной информации. Сбалансированность позволяет гарантировать логарифмическую сложность алгоритма поиска в таких деревьях. Кроме того, желательно, чтобы и операции добавления/удаления узлов дерева также выполнялись эффективно. Рассмотренные рандомизированные, AVL- и красно-черные деревья предлагают решения, в которых добавление и удаление требуют выполнения локальных операций – поворотов, перекраски (в случае красно-черных деревьев), в которые оказываются вовлеченными только отдельные узлы дерева. В итоге эти деревья хотя и оказываются в среднем не идеально сбалансированными, но позволяют реализовать основные операции алгоритмами с оценкой O(log(n)).

Материально-техническое обеспечение работы

Компьютеры с установленным программным обеспечением:

1. Язык программирования высокого уровня.
2. IDE для работы с проектами на выбранном языке программирования.
3. Компьютеры и программное обеспечение для работы с выбранными IDE.

Порядок выполнения работы

1. Ознакомиться с теоретическим материалом.

2. Выполнять задания в порядке их следования так как при выполнении текущего задания потребуются результаты, полученные при выполнении предыдущих заданий.

Контрольные вопросы

1. Что такое идеально сбалансированное двоичное дерево поиска?
2. Что такое рандомизированное двоичное дерево поиска?
3. Что такое AVL- дерево поиска?
4. Что такое красно-черное дерево?
5. Как выполняется правый поворот в двоичном дереве поиска?
6. Как выполняется левый поворот в двоичном дереве поиска?
7. Как использовать повороты для добавления нового узла в корень дерева?
8. С какой вероятностью новый узел будет добавлен в рандомизированное дерево в качестве корня?
9. Выполняется правый поворот для узла A. Который из потомков узла A поднимется на уровень выше в дереве?
10. Выполняется правый поворот для узла A. Который из потомков узла A поднимется на уровень выше в дереве?
11. Какие значения может принимать параметр баланса в AVL-дереве?
12. Какой поворот в AVL-дереве необходимо выполнить для его балансировки, если после добавления нового узла в некотором узле параметр баланса стал равен 2, а в его левом поддереве 1?

13. Какой поворот в AVL-дереве необходимо выполнить для его балансировки, если после добавления нового узла в некотором узле параметр баланса стал равен 2, а в его левом поддереве -1?

14. Какой поворот в AVL-дереве необходимо выполнить для его балансировки, если после добавления нового узла в некотором узле параметр баланса стал равен -2, а в его левом поддереве -1?

15. Какой поворот в AVL-дереве необходимо выполнить для его балансировки, если после добавления нового узла в некотором узле параметр баланса стад равен -2, а в его левом поддереве 1?

16. Что такое черная высота в красно-черном дереве?

17. В чем заключается левый-левый вариант разбалансировки красно-черного дерева?

18. В чем заключается правый-правый вариант разбалансировки красно-черного дерева?

19. В чем заключается левый-правый вариант разбалансировки красно-черного дерева?

20. В чем заключается правый-левый вариант разбалансировки красно-черного дерева?

Перечень использованных информационных ресурсов

1. Р.Сэджвик "Алгоритмы на C++". – М.: ООО "И.Д.Вильямс". – 2016, 1056 с.
2. П.Луридас "Алгоритмы для начинающих: теория и практика для разработчика". – М.: Эксмо. – 2018, 608 с.
3. А.Левитин "Алгоритмы: введение в разработку и анализ". – М.: ООО "И.Д. Вильямс". – 2018, 576 с.
4. Н.Вирт "Алгоритмы и структуры данных". – М.: ДМК Пресс. – 2016, 272с.
5. "Алгоритмы в задачах и примерах: самые эффективные способы и решения от сообщества Stack Overflow". – М.: Издательство АСТ. – 2024, 256 с.
6. А.В.Ахо, Д.Д.Ульман, Д.Э.Хопкрофт "Структуры данных и алгоритмы". – М.: ООО "И.Д. Вильямс". – 2016, 400 с.
7. Т.Х.Кормен, Ч.И.Лейзерсон, Р.Л.Ривест, К.Штайн "Алгоритмы: построение и анализ", 3-е изд. – СПб.: ООО "Диалектика". – 2019, 1328 с.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.